

P2P Secure Transfer Engine

ROOT CAUSE ANALYSIS (RCA) & TECHNICAL ARCHITECTURE SPECIFICATION

Incident Subject: Render 5 GB Bandwidth Exhaustion

Date of Analysis: August 13, 2026

Impact Severity: CRITICAL Server Quota Exhausted

Root Cause: Silent WebRTC Fallback to Socket.IO Relay

1. Executive Summary & Root Cause Analysis (RCA)

 **Primary Incident Report**

The Render free monthly bandwidth quota of **5.0 GB** was fully exhausted in a short time frame despite the application being architected as a peer-to-peer (P2P) file transfer tool. Analysis confirms that **100% of large file transfers were being routed through the Render Node.js server** via Socket.IO WebSocket relay rather than streaming peer-to-peer over WebRTC DataChannels.

Key Technical Findings

- Finding 1: Silent Fallback to Server Relay (relay-file-data)**
In `useWebRTC.ts`, when the WebRTC DataChannel state is not `"open"`, the transport method automatically calls `socket.emit("relay-file-data", payload)`. If WebRTC fails to establish (e.g. across 5G cellular CG-NAT to Home Wi-Fi NAT), all file chunks (64 KB buffers) are continuously sent to the Render server, which relays them to the receiver.
- Finding 2: Ingress + Egress Bandwidth Multiplication (2x Factor)**
When a file is relayed through Render:
 - Sender uploads file to Render server = **2.69 GB Ingress**
 - Render server forwards file to Receiver = **2.69 GB Egress****Total Render Bandwidth Consumed = 5.38 GB for a single 2.69 GB file!** One large movie transfer alone exceeded Render's 5 GB free monthly limit.
- Finding 3: Lack of WebRTC Re-negotiation / ICE Restart on Screen Lock**
When a mobile device locks or switches apps, the OS pauses WebRTC threads and resets `dataChannelRef.current = null`. When screen unlocks, the code restored session state but did NOT re-trigger ICE negotiation. As a result, subsequent chunks permanently streamed over the WebSocket relay.
- Finding 4: CG-NAT Traversal Failure without Working TURN Relay**
Public Google STUN servers (`stun.l.google.com:19302`) only assist direct UDP hole punching when both peers are on simple NATs. When one peer is on Mobile 4G/5G Carrier-Grade NAT (CG-NAT), direct P2P connection fails. Without a dedicated COTURN TURN server configured, WebRTC fails and forces fallback to Render relay.

2. Render Bandwidth Consumption Breakdown Matrix

Transfer Mode	WebRTC DataChannel State	Render Bandwidth Impact	Network Route Path
Pure P2P Mode	OPEN (Direct UDP/SCTP)	0.00 MB / transfer (Only ~2 KB signaling metadata)	Device A → Local Router / 5G → Device B (Direct)
Server Relay Mode (Fallback)	CLOSED / NULL / STALLED	200% of File Size (2.69 GB file = 5.38 GB bandwidth)	Device A → Render Node.js Ingress → Render Egress → Device B

3. Complete System Architecture & Data Flow Diagrams

Diagram 1: End-to-End System Component Architecture



Diagram 2: WebRTC Signaling, Handshake & Data Transfer Sequence

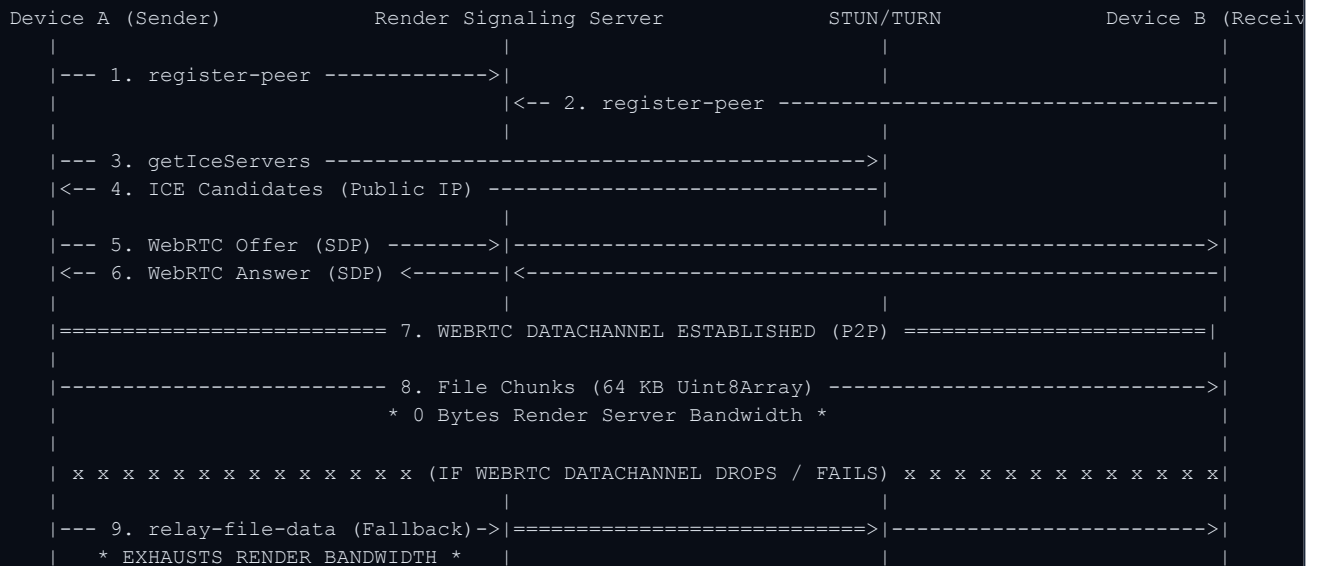
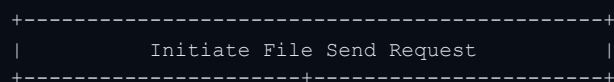
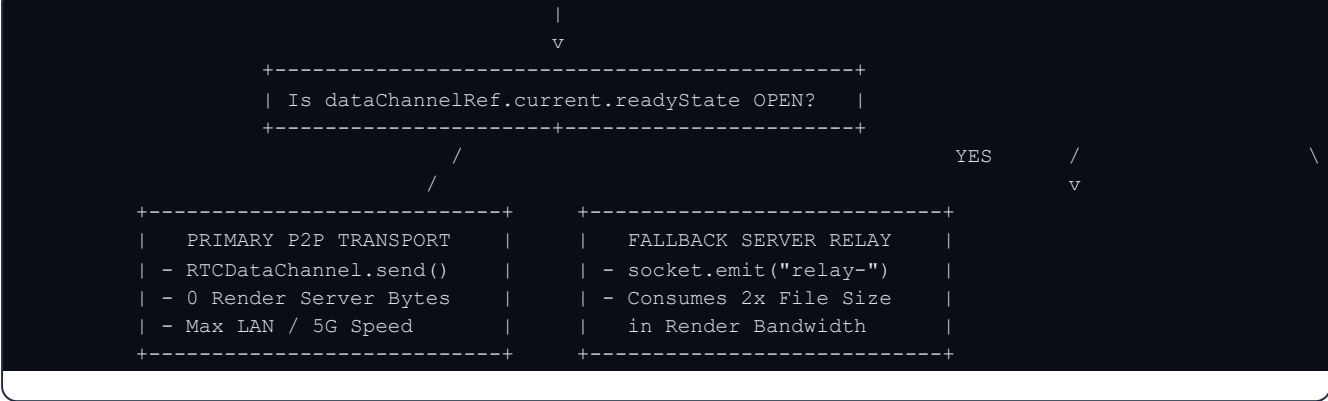


Diagram 3: WebRTC State Machine & Transport Decision Tree





4. Foolproof Strategic Recommendations & Action Plan

🎯 Architecture Remediation Roadmap to Guarantee Zero Server Bandwidth Exhaustion:

Step 1: Implement Mandatory WebRTC ICE Restart on DataChannel Drop

Instead of defaulting permanently to Socket.IO relay when `dataChannelRef.current` is not open, the client must automatically trigger `pc.createOffer({ iceRestart: true })` to re-establish the direct P2P connection automatically while the transfer is paused.

Step 2: Provision a Dedicated Free COTURN Relay Server

Configure a dedicated TURN server (e.g., via Metered.ca or self-hosted COTURN on Oracle Free Tier) so that cross-network connections (Mobile 5G CG-NAT → Home Broadband Wi-Fi) use TURN relaying over Coturn rather than routing binary chunks through Render Socket.IO.

Step 3: Server Relay Rate Limiting & User Notification Warning

Add a visual warning badge in the client UI whenever transfer mode degrades to "Server Relay Mode". If a file exceeds 100 MB in Server Relay Mode, prompt the user or throttle relay speeds to protect server bandwidth quota.

Step 4: Host Static Client Bundles on Vercel / Cloudflare Pages

Host the frontend assets on Vercel or Cloudflare Pages (which offer unlimited bandwidth). Keep Render strictly as a lightweight signaling server (~10 MB/month total bandwidth usage for WebSockets metadata).